

Synthèse du chapitre 3 : Analyse Syntaxique

Ce chapitre explore la phase **centrale** du frontend d'un compilateur : l'**analyse syntaxique** (ou parsing), qui vérifie la structure grammaticale du code source et construit généralement un **arbre de dérivation syntaxique** (ou AST).

Structure du chapitre

1. Définition / Intuition

- L'analyse syntaxique consiste à **vérifier que la séquence de tokens respecte la grammaire** du langage.
- C'est une **étape de validation structurée**, entre l'analyse lexicale et l'analyse sémantique.

2. Spécification des grammaires

- Introduction des **grammaires formelles**, notamment les **grammaires non-contextuelles (CFG)** utilisées en compilation.
- Définitions :
 - **Terminales** (mots du langage),
 - **Non-terminales** (catégories syntaxiques),
 - **Symbole de départ**,
 - **Règles de production**.

3. Grammaire pour les expressions arithmétiques

- Exemple classique : construction d'expressions comme $3 + 4 * 5$.
- Utilisation d'un **arbre de dérivation** pour illustrer la manière dont les règles produisent une expression valide.
- Discussion sur la **priorité des opérateurs** et l'**associativité**.

4. Ambiguïtés grammaticales

- Cas de **grammaires ambiguës** : une même expression peut avoir plusieurs arbres de dérivation.
- Exemple : l'ambiguïté du `if-else` (dangling else).
- Conflits de type **shift/reduce, reduce/reduce**, etc.
- Importance de **résolution via la priorité et l'associativité**.

5. Méthodes d'analyse syntaxique

- **Analyse descendante** (top-down) vs **analyse ascendante** (bottom-up)
- Introduction des **automates à pile**.
- L'analyse ascendante (de type LR) est la plus utilisée dans les compilateurs modernes.

6. Implémentation pratique

- Utilisation de **Ocamlyacc** (équivalent OCaml de Yacc/Bison).
- Structure du parseur :
 - Fichier `.mly` : définition des règles + actions sémantiques
 - Fichier `.mll` : lexer (analyse lexicale)
 - `main.ml` : point d'entrée du programme
- Deux **exemples pratiques** sont donnés :
 - Exemple 1 : grammaire + code + analyse
 - Exemple 2 : grammaire plus complexe (expressions + if/else)

7. Formes récursives

- **Récursivité gauche et récursivité droite**
 - La récursivité gauche est problématique pour les parseurs descendantes.
 - Des techniques d'élimination de la récursivité sont mentionnées.

Concepts clés à retenir

1. **L'analyse syntaxique est une vérification de structure**, basée sur une **grammaire formelle (CFG)**.
2. **Les expressions arithmétiques illustrent bien la construction d'arbres syntaxiques** :
 - l'ordre des opérations est codé dans la grammaire,
 - les **AST permettent de simplifier la représentation** d'une expression.
3. **Les grammaires ambiguës doivent être évitées ou résolues** avec :
 - des règles de priorité/associativité,
 - des indications explicites pour le parseur (ex : `%left`, `%right` en Yacc/Bison/Ocamlyacc).
4. **Il existe deux grandes méthodes de parsing** :
 - **Descendante** (facile à comprendre, utilisée pour les parseurs manuels),
 - **Ascendante** (plus robuste, utilisée dans les outils automatiques type LR/LALR).
5. **La phase de parsing est liée aux automates à pile**, notamment les analyseurs LR.
6. **Les outils comme Ocamlyacc permettent de générer automatiquement un parseur** à partir d'une grammaire.

7. **Un analyseur syntaxique est couplé avec un analyseur lexical**, ce qui permet de combiner reconnaissance des tokens et des structures.